HKUST – Department of Computer Science and Engineering

COMP3711: Design and Analysis of Algorithms – Fall 2026

Practice Midterm A (new questions in the style of the 2016 to 2026 papers)

Time: 2 hours Closed book, one A4 cheat sheet No calculator

Instructions. Answer all questions. Algorithms may be given in pseudocode or plain language; define the input and output of every subroutine. You may use any algorithm from class, homework or tutorials as a black box. Correctness proofs are not required unless asked, but every running time must be justified briefly.

Question	1	2	3	4	Total
Points	10	12	18	20	60

1. Asymptotic analysis [10 points]

(a) [5 points] Algorithms A and B have running times $T_A(n)$ and $T_B(n)$. Fill the last column with "A" (A is always faster for large n), "B", or "U" (not enough information).

Case	$T_A(n)$	$T_B(n)$	Faster
1	$\Theta(n^{1.5} \log n)$	$\Theta(n^2 / (\log n)^3)$	
2	$O(n \log n)$	$\Omega(n)$	
3	$\Theta(4^{\log_2 n})$	$\Theta(n^2 \log n)$	
4	$\Theta(\Sigma_{i=1..n}\, i \cdot \log i)$	$\Theta(n^2)$	
5	$\Theta(3^n)$	$\Theta(2^{2n})$	

(b) [5 points] Derive the exact asymptotic solution of the recurrence below **from first principles** (repeated expansion or recursion tree). Do **not** use the Master Theorem. Assume n is a power of 2. Give the answer in the form $\Theta(n^{\alpha} (\log n)^{\beta})$.

$$T(1) = 1; \quad T(n) = 8 \cdot T(n/2) + n^2 \text{ for } n > 1.$$

2. Randomization [12 points]

A restaurant has n tables. Each of n guests independently chooses a table uniformly at random (several guests may choose the same table).

(a) [4 points] Derive the exact expected number of tables chosen by **nobody**.

(b) [4 points] Derive the exact expected number of tables chosen by **exactly one** guest.

(c) [4 points] Now consider the coupon collector setting with n coupon types. You already hold n/2 distinct types. Derive the expected number of further boxes needed to complete the collection, in the form $\Theta(n^{\alpha} (\log n)^{\beta})$. Argue both the upper and the lower bound.

3. Searching [18 points]

An array A[1..n] of integers is **bitonic**: it is strictly increasing up to some unknown position p and strictly decreasing after it ($1 \leq p \leq n$). Example: A = [3, 7, 12, 20, 15, 9, 4] with p = 4.

(a) [6 points] Design an O(log n) algorithm that finds p. Explain why it is correct and why it runs in O(log n) time.

(b) [6 points] Using (a), design an O(log n) algorithm that decides whether a given value x occurs in A, and if so returns one index where it occurs.

(c) [6 points] Now suppose A[1..n] is sorted in non-decreasing order with possible repeats. Design an O(log n) algorithm that returns the **number of occurrences** of x in A. Give pseudocode for any subroutine you use that was not covered in class.

4. Heaps [20 points]

(a) [8 points] Insert 6, 2, 9, 1, 5, 3, 8, 4 in this order into an initially empty **min-heap**. Draw the tree after each insertion (intermediate swaps need not be shown). Also write the array representation after the last insertion.

(b) [12 points] You are given a min-heap H with n elements in array form H[1..n], and a threshold value x. Design an algorithm that outputs **every** element of H that is strictly smaller than x, in O(k) time, where k is the number of such elements (k is not known in advance and may be 0). The heap must not be modified. Explain the correctness of your algorithm and prove the O(k) bound.

Practice Midterm A – Solutions

1. Asymptotic analysis

(a) 1: **A**. $n^{1.5}$ log n versus $n^2/\log^3 n$: ratio is $\log^4 n / n^{0.5} \to 0$, so A is smaller.
2: **U**. A could be $\Theta(1)$, B could be $\Theta(n^3)$; or A could be $\Theta(n \log n)$, B could be $\Theta(n)$. Nothing is decided.
3: **A**. $4^{\log_2 n} = n^{\log_2 4} = n^2$, which is smaller than n^2 log n.
4: **B**. $\Sigma\, i \log i \geq \Sigma_{i \geq n/2} (n/2) \log(n/2) = \Theta(n^2 \log n)$, and $\leq n \cdot n \log n$, so A = $\Theta(n^2 \log n)$ is slower than n^2.
5: **A**. $2^{2n} = 4^n$; $(3/4)^n \to 0$.

(b) Expansion:
$T(n) = 8T(n/2) + n^2 = 8[8T(n/4) + (n/2)^2] + n^2 = 8^2T(n/4) + 2n^2 + n^2$
$= 8^3T(n/8) + 4n^2 + 2n^2 + n^2 = \ldots = 8^k\, T(n/2^k) + n^2(1 + 2 + 4 + \ldots + 2^{k-1})$.
Level i costs $8^i \cdot (n/2^i)^2 = 2^i\, n^2$: an increasing geometric series, so the last level dominates. Stop at $k = \log_2 n$: $8^k = n^3$ leaves of cost 1, and the sum is $n^2(2^k - 1) = n^2(n - 1)$.
$T(n) = n^3 + n^3 - n^2 =$ **$\Theta(n^3)$** ($\alpha = 3$, $\beta = 0$).

2. Randomization

(a) Let $X_j = 1$ if table j is chosen by nobody. Each guest avoids table j with probability $1 - 1/n$, independently, so $\Pr[X_j = 1] = (1 - 1/n)^n$. By linearity, $E[\text{empty tables}] = \Sigma_j\, E[X_j] =$ **$n(1 - 1/n)^n$** (about n/e).

(b) Let $Y_j = 1$ if exactly one guest chooses table j. There are n choices for which guest, that guest picks j with probability 1/n, the other $n - 1$ avoid it with probability $(1 - 1/n)^{n-1}$. So $\Pr[Y_j = 1] = n \cdot (1/n) \cdot (1 - 1/n)^{n-1} = (1 - 1/n)^{n-1}$. E = **$n(1 - 1/n)^{n-1}$** (also about n/e).

(c) When i distinct types are held, each box is new with probability $(n - i)/n$, so the expected wait is $n/(n - i)$. Summing from $i = n/2$ to $n - 1$:
$E = n(1/(n/2) + 1/(n/2 - 1) + \ldots + 1/1) = n \cdot H_{n/2}$.
Upper bound: $H_{n/2} \leq 1 + \log_2(n/2) = O(\log n)$ by the doubling-block argument (each block sums to at most 1). Lower bound: $H_{n/2} \geq (1/2) \log_2(n/2)$ because each block sums to at least 1/2. Hence E = **$\Theta(n \log n)$** ($\alpha = 1$, $\beta = 1$). Contrast with the first n/2 coupons, which cost only $\Theta(n)$.

3. Searching

(a) Peak(A, l, r): input a bitonic subarray containing the peak; output p.
If l = r return l. Let $m = \lfloor (l + r)/2 \rfloor$. If $A[m] < A[m + 1]$ the array is still increasing at m, so the peak is to the right: return Peak(A, m + 1, r). Otherwise $A[m] > A[m + 1]$ (distinct neighbours), so the peak is at m or to its left: return Peak(A, l, m).

Correctness: on an increasing stretch the peak lies strictly right of m; on a decreasing stretch it lies at or left of m; the chosen half always contains p. Each call halves the range and does O(1) work, so O(log n).

(b) Compute p in O(log n). A[1..p] is strictly increasing: run ordinary binary search for x there, O(log n). A[p + 1..n] is strictly decreasing: run binary search with the comparison reversed (if x > A[m] go left, else go right), O(log n). Return the first index found, or "absent". Total O(log n).

(c) Two searches for boundaries. First(A, x) returns the smallest index i with A[i] ≥ x (n + 1 if none); Last(A, x) returns the smallest index with A[i] > x. Count = Last − First (0 if x is absent, since both return the same index).

```
First(A, x):                          // smallest i with A[i] >= x
  l = 1; r = n + 1
  while l < r:
    m = floor((l + r) / 2)
    if A[m] < x: l = m + 1 else r = m
  return l
Last(A, x): identical with the test "A[m] <= x".
```

Each loop halves r − l and does O(1) work, so each search is O(log n); two searches give O(log n). This is the HW1 lower-bound binary search and the S25/S26 "frequency" idea without the Θ(log n)-distinct assumption.

4. Heaps

(a) Arrays after each insertion (bubble up while smaller than the parent):
[6] → [2, 6] → [2, 6, 9] → [1, 2, 9, 6] → [1, 2, 9, 6, 5] → [1, 2, 3, 6, 5, 9] → [1, 2, 3, 6, 5, 9, 8] → [1, 2, 3, 4, 5, 9, 8, 6].
Trees: each array drawn level by level (positions 1 | 2 3 | 4 5 6 7 | 8). Final tree: root 1; children 2, 3; grandchildren 4, 5, 9, 8; leaf 6 under 4. Half a point off per wrong tree in the real paper, so draw all eight.

(b) Report(i): input an index i of H; output every element in the subtree of i that is smaller than x.
If i > n or H[i] ≥ x: return. Otherwise output H[i], then Report(2i) and Report(2i + 1). Top-level call Report(1).
Correctness: by the heap property every element in the subtree of i is at least H[i]. So if H[i] ≥ x, nothing in that subtree is smaller than x and pruning is safe; if H[i] < x we output it and continue, so no element smaller than x is missed, and nothing larger is ever printed.
Running time: every call that outputs an element makes two child calls. Calls that do not output (H[i] ≥ x or i > n) do O(1) work and stop. The number of non-output calls is at most 2k (two per output call) plus one for the root when k = 0. Total calls ≤ 2k + 1, each O(1), so O(k). The heap is only read.